

An Introduction to Category Theory for Machine Learning

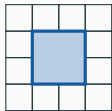
Detagnon Bernic Gbaguidi

October 2026

School of Mathematics and Statistics , RIT

Framing

Motivation: a zoo of architectures



Images



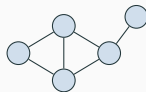
CNN



Sequences



RNN



Graphs



GNN



Sets

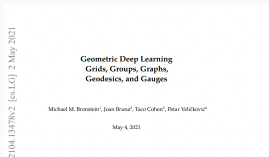


Deep Sets /
Transformers

- Each architecture was found largely by **intuition, analogy, and trial and error**.
- For a long time there was **no unifying theory** of *why* an architecture suits its data.

Given data with some structure, which architectures make sense – and why?

Toward a mathematical theory of architectures



Geometric Deep Learning (Bronstein et al., 2021)
symmetries of the data $\Rightarrow$ CNNs, GNNs,
Transformers
arXiv:2104.13478



Categorical Deep Learning (Gavranović et al., ICML 2024)
one categorical theory for constraints *and*
implementations
arXiv:2402.15332

Where we are, and where we are heading

- **Today:** given the structure of the data, theory tells us which architectures **respect** it.
- **Toward:** *prescribing* the right architecture for a setting from mathematics rather than search – an open research direction.

What this talk does

To reason about architectures mathematically, we first need a **precise language for models and training**. Today's talk answers three questions:

1. **What is a machine learning model, categorically?**
2. **What is backpropagation, categorically?**
3. **What does this framework give us that we did not have before?**

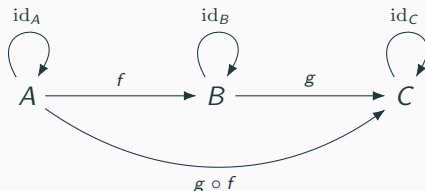
The foundation on which architecture-level theory is being built.

The categorical setting

Category: a reminder

A **category** $\mathcal{C}$ consists of:

- Objects $A, B, C, \dots$ and morphisms $f : A \rightarrow B$.
- Associative composition $\circ$ with identity morphisms id_A .



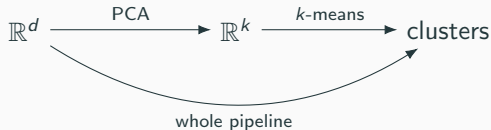
Every object has an identity: $f \circ \text{id}_A = f = \text{id}_B \circ f$.

Familiar examples: **Set**, **Vect**_k, **Smooth** (Euclidean spaces, smooth maps).

Example: the category of datasets

A category you already work in every day:

- **Objects:** feature spaces / datasets — $\mathbb{R}^d$, a set of records, a label set.
- **Morphisms** $A \rightarrow B$: data transformations — normalization, PCA projection, an encoding, a feature map.
- **Composition:** chaining transformations.
- **Identity:** the pass-through transformation.



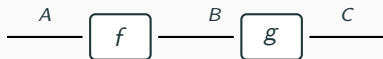
A preprocessing pipeline *is* composition in this category.
(`sklearn.Pipeline([pca, kmeans]) = kmeans  $\circ$  pca.`)

String diagrams: a proof calculus

A morphism $f : A \rightarrow B$ is drawn as a box with a wire in and a wire out:



Composition chains left-to-right:



Sound and complete calculus for morphisms in monoidal categories (Joyal–Street).

Functors: maps between categories

A **functor** $F : C \rightarrow D$ sends objects to objects and morphisms to morphisms, preserving structure:

$$F(\text{id}_A) = \text{id}_{F(A)}, \quad F(g \circ f) = F(g) \circ F(f).$$

The diagram illustrates the mapping of a morphism by a functor. On the left, in category C , there is a morphism f from object A to object B . On the right, in category D , there is a morphism $F(f)$ from object $F(A)$ to object $F(B)$. A curved arrow labeled F points from the morphism f to the morphism $F(f)$, indicating the mapping of the morphism by the functor F .

A functor is a “structure-preserving map” of categories — the categorical analogue of a homomorphism.

Example: one-hot encoding is a functor

One-hot encoding is a functor $F : \mathbf{FinSet} \rightarrow \mathbf{Vect}_{\mathbb{R}}$.

- **On objects:** a label set $S \mapsto \mathbb{R}^{|S|}$, one basis vector per label:

$$\{\text{cat}, \text{dog}, \text{bird}\} \mapsto \mathbb{R}^3, \quad \text{cat} \mapsto (1, 0, 0), \quad \text{dog} \mapsto (0, 1, 0), \quad \text{bird} \mapsto (0, 0, 1).$$

- **On morphisms:** a function between label sets $\mapsto$ a matrix. Merging labels, h : cat, dog $\mapsto$ mammal; bird $\mapsto$ bird, gives

$$F(h) = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad F(h)(0.2, 0.5, 0.3) = (0.7, 0.3).$$

It **sums the scores of merged classes**.

Functoriality: merge again, k : mammal, bird $\mapsto$ animal, so $F(k) = \begin{pmatrix} 1 & 1 \end{pmatrix}$:

$$F(k \circ h) = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix} = F(k) F(h), \quad F(\text{id}_S) = I.$$

Composing label maps = multiplying their matrices.

Natural transformations: maps between functors

Given two functors $F, G : C \rightarrow D$, a **natural transformation** $\alpha : F \Rightarrow G$ assigns to each object A a morphism $\alpha_A : F(A) \rightarrow G(A)$, such that for every $f : A \rightarrow B$ the square commutes:

$$\begin{array}{ccc} F(A) & \xrightarrow{\alpha_A} & G(A) \\ F(f) \downarrow & & \downarrow G(f) \\ F(B) & \xrightarrow{\alpha_B} & G(B) \end{array}$$

$$\alpha_B \circ F(f) = G(f) \circ \alpha_A.$$

A “uniform” way to turn F -structure into G -structure, respecting all morphisms.

Example: a natural transformation of encodings

Two functors $F, G : \mathbf{FinSet} \rightarrow \mathbf{Vect}_{\mathbb{R}}$ — say F = one-hot, G = some other embedding of the same labels.

A **natural transformation** $\alpha : F \Rightarrow G$ gives, for each label set S , a linear map $\alpha_S : F(S) \rightarrow G(S)$, commuting with relabelings $h : S \rightarrow T$:

$$\begin{array}{ccc} F(S) & \xrightarrow{\alpha_S} & G(S) \\ F(h) \downarrow & & \downarrow G(h) \\ F(T) & \xrightarrow{\alpha_T} & G(T) \end{array}$$

“Natural” = the embedding is *choice-free*: it transforms consistently when you rename or merge labels.
No naturality $\Rightarrow$ an arbitrary, incompatible choice at each label set.

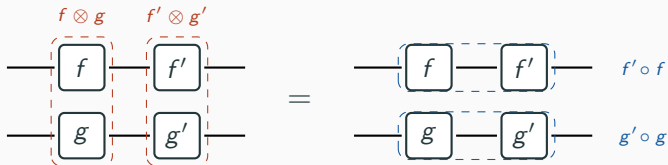
Monoidal category: the tensor of morphisms

A **monoidal category** $(C, \otimes, I)$ adds a second way to combine: a bifunctor $\otimes : C \times C \rightarrow C$.

Two morphisms $f : A \rightarrow B$ and $g : C \rightarrow D$ combine into $f \otimes g : A \otimes C \rightarrow B \otimes D$ — run *in parallel*:



Interchange law: composing-then-tensoring = tensoring-then-composing.



Monoidal category: the unit and coherence

The **unit** I is a “neutral” object for $\otimes$: tensoring with it changes nothing (up to iso). In string diagrams, a wire labeled I is *invisible*.

$$\begin{array}{c} I \\ \text{---} \\ A \\ \text{---} \end{array} = \text{---}^A$$

Special morphisms interact with I :

$$\begin{array}{cc} \text{state } I \rightarrow A: & \boxed{a} \text{---}^A \\ \text{effect } A \rightarrow I: & \text{---}^A \boxed{e} \end{array}$$

Symmetric Monoidal category

A monoidal category is **symmetric** if it comes with a way to *swap* tensor factors:

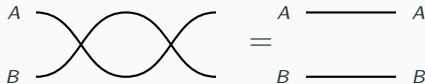
$$\sigma_{A,B} : A \otimes B \xrightarrow{\sim} B \otimes A,$$

natural in A and B , drawn as **crossing wires**:



The defining condition: **swapping twice does nothing**.

$$\sigma_{B,A} \circ \sigma_{A,B} = \text{id}_{A \otimes B}$$



Examples **(Set, $\times$)**, **(Vect_k, $\otimes_k$)**, **(Smooth, $\times$)** are all symmetric.

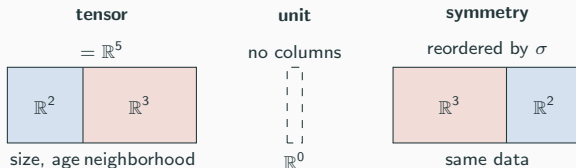
Example: feature concatenation is symmetric monoidal

Work in $(\mathbf{Vect}, \times)$: objects are feature spaces, the tensor $\otimes$ is **concatenating feature blocks**. A house record has a numeric part and a categorical part.

- **Tensor.** $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ standardizes (size, age); $g : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ is the one-hot neighborhood block. In parallel:

$$f \otimes g : \mathbb{R}^2 \times \mathbb{R}^3 \rightarrow \mathbb{R}^2 \times \mathbb{R}^3 = \mathbb{R}^5 \quad \text{— the full row.}$$

- **Unit.** $\mathbb{R}^0$ is the **empty block**: adding no columns changes nothing, $\mathbb{R}^0 \times \mathbb{R}^m \cong \mathbb{R}^m$.
- **Symmetry.** $\mathbb{R}^m \times \mathbb{R}^n \cong \mathbb{R}^n \times \mathbb{R}^m$ just **reorders columns** — swap twice and you are back.



Building a design matrix *is* working in a symmetric monoidal category: concatenate blocks, the empty block is neutral, block order is irrelevant

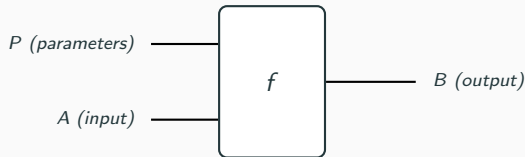
Machine learning models are morphisms

Observation (Shiebler et al., Section 2.1.2)

A machine learning model is a morphism

$$f : P \otimes A \rightarrow B$$

in a symmetric monoidal category $\mathcal{C}$.



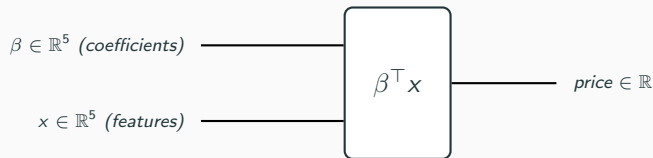
Question 1 answered.

Example: linear regression is a morphism

Put a model on the features we just built. The simplest one: **linear regression**.

$$f : P \otimes A \rightarrow B, \quad f(\beta, x) = \beta^\top x.$$

- $A = \mathbb{R}^5$ — the feature row (size, age, neighborhood one-hot).
- $P = \mathbb{R}^5$ — the coefficients β (the **parameters**).
- $B = \mathbb{R}$ — the predicted price.



The coefficient-vs-feature split you already make *is* the parameter-vs-input split $P \otimes A$.

Training (finding β) is a state $I \rightarrow P$ — and the rest of this talk is **how to find it**.

Backpropagation in Category Theory

The claim

In a reverse derivative category, the reverse derivative R applied to a **composition** of morphisms **is the backpropagation algorithm.**

The rest of this arc earns that sentence: define the structure, exhibit it in **Smooth**, and match it to textbook backprop.

Ingredient: a reverse derivative category

A **reverse derivative category** is a Cartesian left-additive category with an operator

$$R : \{f : A \rightarrow B\} \longrightarrow \{R[f] : A \times B \rightarrow A\}$$

satisfying **seven axioms** (RDC.1–7). The key one is

RDC.5 – the reverse chain rule (element-free)

$$R[g \circ f] = R[f] \circ \langle \pi_1, R[g] \circ \langle f \circ \pi_1, \pi_2 \rangle \rangle$$

Axioms RDC.1–7: Cockett–Cruttwell–Gallagher–Lemay (CSL 2020); surveyed in Shiebler et al. (2021, §2.2).

Smooth is a reverse derivative category

In **Smooth** objects are Euclidean spaces, so we *may* evaluate at points.

$$R[f](x, y') = J_f(x)^\top y' \quad (\text{transpose Jacobian} \times \text{cotangent}).$$

The abstract RDC.5 becomes, at a point (x, z') :

$$R[g \circ f](x, z') = R[f](x, R[g](f(x), z')).$$

Why it holds: the classical chain rule $J_{g \circ f}(x) = J_g(f(x)) J_f(x)$, transposed, gives

$$J_{g \circ f}(x)^\top = J_f(x)^\top J_g(f(x))^\top,$$

which is exactly the nested reverse form above.

R of a composition = backprop

Network with a scalar loss:

$$x \xrightarrow{f} h \xrightarrow{g} y \xrightarrow{L} \mathbb{R}, \quad h = f(x), \quad y = g(h).$$

Goal: $\frac{\partial L}{\partial x}$.

1. **Backprop** — cotangents, right to left:

$$\bar{y} = \nabla L(y), \quad \bar{h} = J_g(h)^\top \bar{y}, \quad \bar{x} = J_f(x)^\top \bar{h}.$$

2. In terms of Jacobians:

$$\frac{\partial L}{\partial x} = \bar{x} = J_f(x)^\top J_g(h)^\top \nabla L(y).$$

3. Replace each $J^\top$ by R (in **Smooth**: $R[m](a, v) = J_m(a)^\top v$, $\nabla L(y) = R[L](y, 1)$):

$$\frac{\partial L}{\partial x} = R[f]\left(x, R[g]\left(h, R[L](y, 1)\right)\right).$$

4. **Collapse with RDC.5** (applied twice):

$$\frac{\partial L}{\partial x} = R[L \circ g \circ f](x, 1).$$

R of a composition = backprop

Conclusion 1

Backprop computes R of the composite network. R of a composition is exactly backprop.

Lenses: backprop is a functor

Ingredient: the simple lens

Definition

A **simple lens** $A \rightleftarrows B$ is a pair (f, f^*) with

$$f : A \rightarrow B \quad (\text{forward}), \quad f^* : A \times B \rightarrow A \quad (\text{backward}).$$

The category $\text{Lens}(C)$

Lenses assemble into a category:

- **Objects:** the objects of C .
- **Morphisms** $A \rightarrow B$: simple lenses (f, f^*) .
- **Identity on** A : (id_A, π_2) .
- **Composition** of $(f, f^*) : A \rightleftarrows B$ and $(g, g^*) : B \rightleftarrows C$:

$$(g, g^*) \circ (f, f^*) = (g \circ f, h),$$

with backward part

$$h = f^* \circ \langle \pi_1, g^* \circ \langle f \circ \pi_1, \pi_2 \rangle \rangle.$$

Compare with RDC.5. **Lens composition is defined to chain backward maps by the reverse chain rule.**

Package each morphism as its lens

Proposition (Shiebler et al., Prop. 1)

Let (C, R) be a reverse derivative category. The assignment

$$R : C \longrightarrow \mathbf{Lens}(C), \quad R(A) = A, \quad R(f) = (f, R[f])$$

is a **functor**.

Identities are preserved ($R[\text{id}] = \pi_2$, the identity lens). The real content is **preservation of composition**:

$$R(g \circ f) \stackrel{?}{=} R(g) \circ R(f).$$

It is a functor – and why that is the point

Compute both sides of $R(g \circ f) = R(g) \circ R(f)$:

- $R(g) \circ R(f)$ — evaluate by the *lens-composition* formula with $f^* = R[f]$, $g^* = R[g]$: backward part $R[f] \circ \langle \pi_1, R[g] \circ \langle f \circ \pi_1, \pi_2 \rangle \rangle$.
- $R(g \circ f)$ — backward part is $R[g \circ f]$.

Same forward part; the backward parts agree **if and only if** RDC.5 holds – and it does. So R is a functor.

Conclusion 2 – the payoff

R is a functor, so it **preserves composition**.

And composition is exactly backprop (Conclusion 1).

To backpropagate a whole network, just apply R to the composite – functoriality runs the backward pass.

Adding parameters

The Para construction

Networks have *parameters* adjusted during training.

Definition ($\mathbf{Para}(C)$)

For a symmetric monoidal C , the bicategory $\mathbf{Para}(C)$ has:

- **0-cells:** objects of C .
- **1-cells** $A \rightarrow B$: pairs (P, f) with $f : P \otimes A \rightarrow B$.
- **2-cells** $(P', f') \Rightarrow (P, f)$: reparameterizations $r : P' \rightarrow P$.
- **Composition:** $(Q, g) \circ (P, f) = (Q \otimes P, g \circ (1_Q \otimes f))$.

Composition tensors parameters. Associative only *up to iso*: hence bicategory.

Unpacking Para(Lens(C))

Apply **Para** to the *base* **Lens(C)**. Peel the two layers:

- **0-cells:** objects of **Lens(C)** = objects of *C*.
- **1-cell** $A \rightarrow B$: a pair (P, Φ) where P is a parameter object and Φ is a **lens** $P \otimes A \rightleftarrows B$ (a morphism of **Lens(C)**).
- **2-cell:** a reparameterization $r : P' \rightarrow P$ – a **morphism of Lens(C)**, i.e. itself a *lens* $P' \rightleftarrows P$.

Unfold the lens $\Phi = (f, f^*)$ inside the 1-cell:

$$\underbrace{f : P \otimes A \rightarrow B}_{\text{forward pass}}, \quad \underbrace{f^* : (P \otimes A) \times B \rightarrow P \otimes A}_{\text{backward pass}}.$$

The backward map lands in $P \otimes A$, so it returns **two gradients**: one for the parameters, one for the input.

A 1-cell of $\mathbf{Para}(\mathbf{Lens}(C))$ is a trainable layer

Read off what each piece of data means for a neural-network layer:

categorical data	neural-network meaning
parameter object P	the layer's weights
forward $f : P \otimes A \rightarrow B$	the forward pass (prediction)
backward f^* , part in P	weight gradient – used to <i>update</i> the weights
backward f^* , part in A	input gradient – <i>passed</i> to the previous layer

Learners
 $\mathbf{Learn}(C) := \mathbf{Para}(\mathbf{Lens}(C)).$

A 1-cell is a trainable layer; composing 1-cells **stacks layers** (parameters tensor together, gradients chain).

What this buys us

Answers to the three questions

1. **What is an ML model?**

A 1-cell in $\mathbf{Para}(\mathbf{Lens}(C))$ for a reverse derivative category C .

2. **What is backpropagation?**

Functoriality of $R : C \rightarrow \mathbf{Lens}(C)$, lifted through $\mathbf{Para}$.

3. **What does the framework give us?**

(Next slide.)

What the framework gives us

The key point: nothing here used real numbers
Every construction only required a **reverse derivative category**.

Smooth was just one example.

In **Smooth**, “backprop is a functor” is the chain rule – nothing new.

But the theorem holds in **every** reverse derivative category. So any setting with that structure gets, *for free*:

- a precise notion of a trainable layer (P, f, f^*) ,
- a correct backward pass for any composite network,
- a separation between the model and its training rule.

Prove once, at the level of structure; apply everywhere that structure exists.

e.g. circuits of logic gates form a reverse derivative category $\Rightarrow$ gradient-style learning for circuits.

Where to go next

- **Categorical probability.** Fritz's Markov categories give the same story with $\mathcal{C}$ a category of stochastic maps.
- **Enriched category theory.** The framework extends to V -enriched categories — opening the door to metric-space, order-theoretic, and quantum-inspired ML.
- **Applied category theory community.** Fong–Spivak, *Seven Sketches in Compositionality*. The ACT conference. The Compositionality journal.

Summary

- Machine learning models are morphisms in a monoidal category.
- Backpropagation is a functor $R : C \rightarrow \mathbf{Lens}(C)$; its functoriality *is* the reverse chain rule.
- The framework is a language, not decoration: it explains what works, constrains what's well-formed, and generalizes to new mathematical settings.

Thank you. Questions?

Backup: why left-additive?

Gradients live in tangent spaces (vector-space-like). Left-additive gives:

- Addition of parallel morphisms $f + g$.
- Zero morphism $0_{A,B}$.
- Distributivity of composition over addition.

This enables the differential axioms (CDC.1–7, RDC.1–7) to be stated categorically.

Backup: Markov categories

Section 3 of the paper: same framework for *categorical probability*:

- C = a Markov category (symmetric monoidal + commutative comonoid per object).
- Morphisms = stochastic maps (Markov kernels).
- Every object has *copy* and *delete*.
- Determinism, conditioning, Bayesian inverses become categorical operations.

“Learners” become probabilistic models with categorically-defined training.

Backup: is this just backprop with fancy notation?

Three answers:

1. Categorical structure identifies correctness conditions ad hoc frameworks don't.
2. It enables transfer of techniques across base categories.
3. It reveals *why* algorithmic combinations work.

The framework doesn't invent new algorithms. It identifies *what makes them work*.